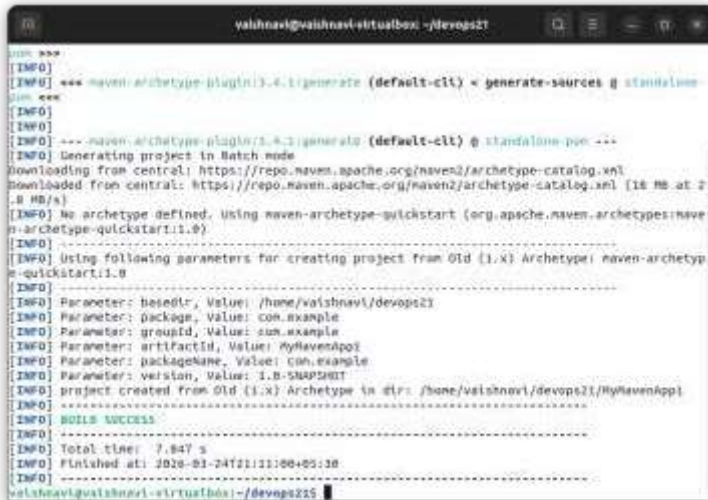


Simple Maven Project

Step1:

To create a basic Maven Java project, run the following command:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```



```

valshnavi@valshnavi-virtualbox: ~/devops21
$ mvn archetype:generate
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone ---
[INFO] Generating project in Batch mode
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml [16 kB at 2.0 MB/s]
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] -----
[INFO] Parameter: basedir, Value: /home/valshnavi/devops21
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/valshnavi/devops21/MyMavenApp
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 7.847 s
[INFO] Finished at: 2020-03-24T22:11:00+05:30
[INFO] -----
valshnavi@valshnavi-virtualbox: ~/devops21

```

The command can be broken down as:

- DgroupId=com.example – project’s base package (reverse-domain style).
- DartifactId=my-maven-app – defines the name of the project (will also be the folder name).
- DarchetypeArtifactId=maven-archetype-quickstart – sets the Maven template to be the basic Java application.
- DinteractiveMode=false – deactivates interactive mode and uses the given values automatically.

Step 2:

Use `$cd` command to go into the directory.

Execute `$tree` command to view the structure of the project

```
vaishnavi@vaishnavi-virtualbox:~/devops21$ cd MyMavenApp1
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp1$ tree
```

```

.
├── pom.xml
└── src
    ├── main
    │   └── java
    │       ├── com
    │       │   └── example
    │       │       └── App.java
    └── test
        └── java
            ├── com
            │   └── example
            │       └── AppTest.java
```

9 directories, 3 files

```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp1$
```

Step 3:

Edit the pom.xml file

Add the required dependencies and plugins required from central maven repository

For given project, we will download unit testing dependency from Jenkins.

```

1 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
3   <modelVersion>4.0.0</modelVersion>
4   <groupId>com.example</groupId>
5   <artifactId>MyMavenApp</artifactId>
6   <packaging>jar</packaging>
7   <version>1.0-SNAPSHOT</version>
8
9
10  <name>MyMavenApp</name>
11  <url>http://maven.apache.org</url>
12
13  <!-- Properties: Customize Java version or plugin versions -->
14  <properties>
15    <maven.compiler.source>11</maven.compiler.source>
16    <maven.compiler.target>11</maven.compiler.target>
17  </properties>
18  <dependencies>
19    <dependency>
20      <groupId>junit</groupId>
21      <artifactId>junit</artifactId>
22      <version>3.8.1</version>
23      <scope>test</scope>
24    </dependency>
25  </dependencies>
26
27  <!-- build: Configuring plugins and build settings -->
28  <build>
29    <plugins>
30      <!-- Example: Maven Compiler Plugin to compile Java code -->
31      <plugin>
32        <groupId>org.apache.maven.plugins</groupId>
33        <artifactId>maven-compiler-plugin</artifactId>
34        <version>3.8.1</version>
35        <configuration>
36          <source>11</source>
37          <target>11</target>
38        </configuration>
39      </plugin>
40    </plugins>
41  </build>
42</project>
```

Step 4:

Execute the command `$mvn compile`

If the command shoes output as “build success”, it means that the project has compiled successfully

```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$ mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenApp >-----
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vaishnavi/devops21/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.713 s
[INFO] Finished at: 2026-03-24T21:20:16+05:30
[INFO]
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$
```

Step 5:

Execute \$tree command again to view the complete structure after adding plugins and dependencies to the pom.xml file

[illegible]

Step 6:

Execute `$mvn test` to run unit tests

```

valshnavi@valshnavi-virtualbox:~/devops21/MyMavenApp$ mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----[ com.example.MyMavenApp ]-----
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/valshnavi/devops21/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/valshnavi/devops21/MyMavenApp/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:2.22.1:test (default-test) @ MyMavenApp ---
[INFO]
[INFO] -----[ T E S T S ]-----
[INFO]
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.303 s
[INFO] Finished at: 2026-03-24T22:27:42+05:30
[INFO]
valshnavi@valshnavi-virtualbox:~/devops21/MyMavenApp$
valshnavi@valshnavi-virtualbox:~/devops21/MyMavenApp$

```

Step 7:

Run mvn package command to generate a JAR application bundle which contains our basic Maven Hello World application

```

valshnavi@valshnavi-virtualbox:~/devops21/MyMavenApp$ mvn package
[INFO] Scanning for projects...
[INFO]
[INFO] -----[ com.example.MyMavenApp ]-----
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/valshnavi/devops21/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/valshnavi/devops21/MyMavenApp/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:2.22.1:test (default-test) @ MyMavenApp ---
[INFO]
[INFO] -----[ T E S T S ]-----
[INFO]
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.003 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- maven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenApp ---
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.468 s
[INFO] Finished at: 2026-03-24T22:28:09+05:30
[INFO]
valshnavi@valshnavi-virtualbox:~/devops21/MyMavenApp$

```

Step 8:

Run the following command to run the JAR file: java -jar target/MyMavenApp-1.0-SNAPSHOT.jar

```
[INFO] -----
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$ java -jar target/MyMavenApp-1.0-SNAPSHOT.jar
HelloClass, happy to see you all
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenApp$ █
```

Pushing The Project into Git:

Execute the commands:

```
$ git config --global user.name "vaishnavim1121"
```

```
$ git config --global user.email "vaishnavim1121@gmail.com"
```

```
$ git init
```

```
$ git add .
```

```
$ git commit -m "initial commit"
```

```
$ git remote add origin https://github.com/vaishnavim1121/MyMavenApp.git
```

```
$ git remote set-url origin https://*PAT*@github.com/vaishnavim1121/MyMavenApp
```

```
$ git push -u origin master
```

```
Activities Terminal Mar 17 10:27
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03

Note: if your new repositories, which will suppress this warning, call:
Note:
Note: git config --global init.defaultBranch main
Note:
Note: Names commonly chosen instead of 'master' are 'main', 'trunk' and
Note: development. The just-created branch can be renamed via this command:
Note:
Note: git branch -m main
Initialized empty Git repository in /home/vaishnavi/devops21/MyMavenSeleniumApp03/.git/
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$ git add
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$ git commit -m "first commit"
[master (root-commit) 1af4a8b] first commit
13 files changed, 285 insertions(+)
create mode 100644 pom.xml
create mode 100644 src/main/java/com/example/App.java
create mode 100644 src/test/java/com/example/AppTest.java
create mode 100644 target/MyMavenSeleniumApp03-1.0-SNAPSHOT.jar
create mode 100644 target/classes/com/example/App.class
create mode 100644 target/maven-archiver/pom.properties
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
create mode 100644 target/surefire-reports/TEST-com.example.AppTest.xml
create mode 100644 target/surefire-reports/com.example.AppTest.txt
create mode 100644 target/test-classes/com/example/AppTest.class
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$ git config --global user.name "vaishnavim1121"
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$ git config --global user.email "vaishnavim1121@gmail.com"
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$ git remote add origin https://github.com/vaishnavim1121/MyMavenSeleniumApp03.git
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$ git remote set-url origin https://ghe_wHhYMP65xgt2QxFOV7NoGjtJal6ZV3WIFI@github.com/vaishnavim1121/MyMavenSeleniumApp03
set-url: command not found
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$ git remote set-url origin https://ghe_wHhYMP65xgt2QxFOV7NoGjtJal6ZV3WIFI@github.com/vaishnavim1121/MyMavenSeleniumApp03
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$ git push -u origin master
remote: repository not found.
fatal: repository 'https://github.com/vaishnavim1121/MyMavenSeleniumApp03/' not found
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$ git remote set-url origin https://ghe_wHhYMP65xgt2QxFOV7NoGjtJal6ZV3WIFI@github.com/vaishnavim1121/MyMavenSeleniumApp03
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$ git push -u origin master
Enumerating objects: 39, done.
Counting objects: 100% (39/39), done.
Delta compression using up to 10 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (39/39), 8.40 KiB | 721.00 KiB/s, done.
Total 39 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/vaishnavim1121/MyMavenSeleniumApp03
 * [new branch] master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp03$
```

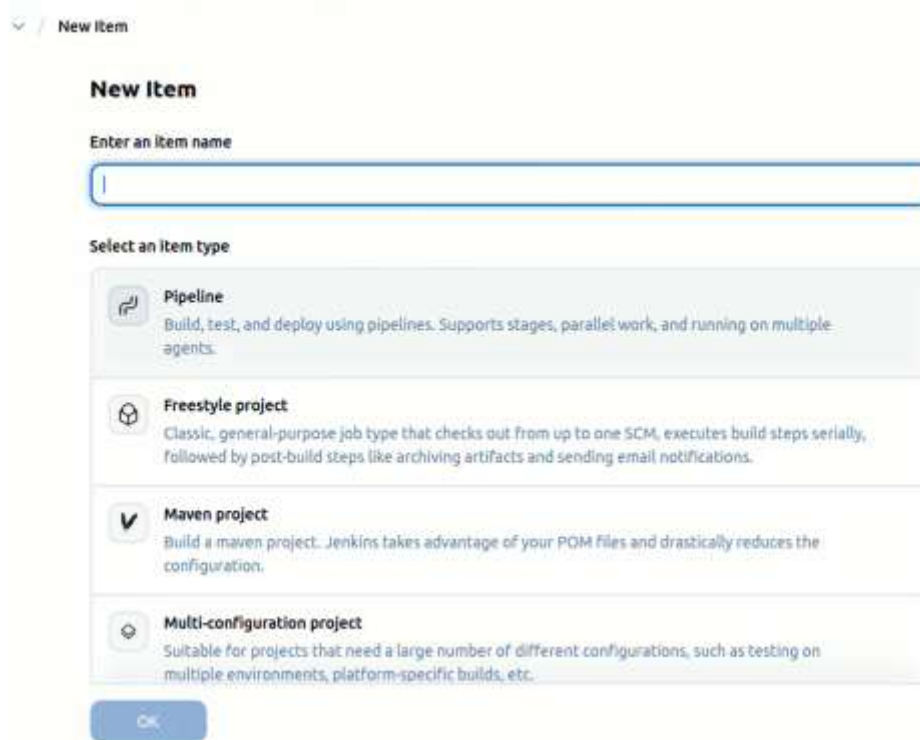

Second Phase

Depicting Jenkins Pipeline

Follow the given steps to deploy the project on Jenkins.

create a Jenkins pipeline job that will use the Jenkinsfile from your project repository

go to the Jenkins home page and click on “New Item” to create a job, enter the item name select Pipeline as the project type.



✓ / New Item

New Item

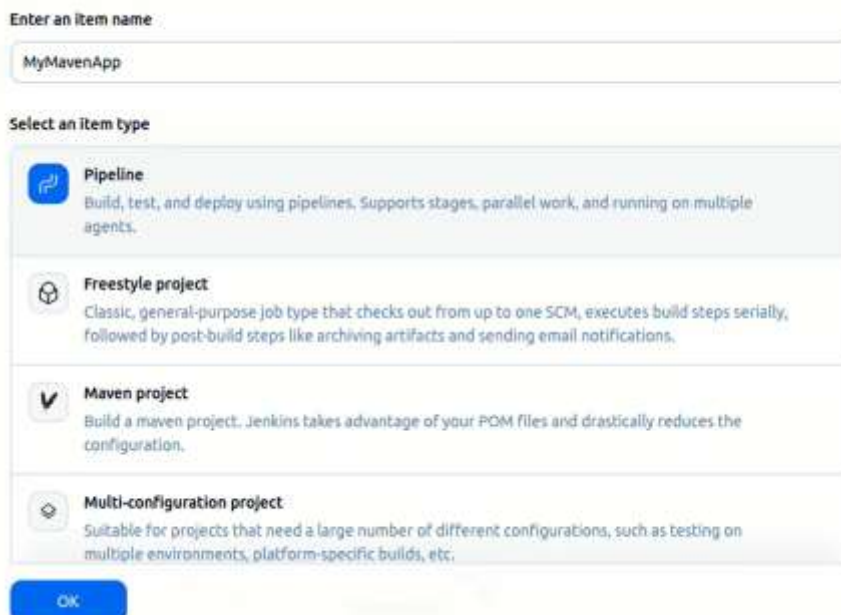
Enter an item name

Select an item type

-  **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

OK

New Item



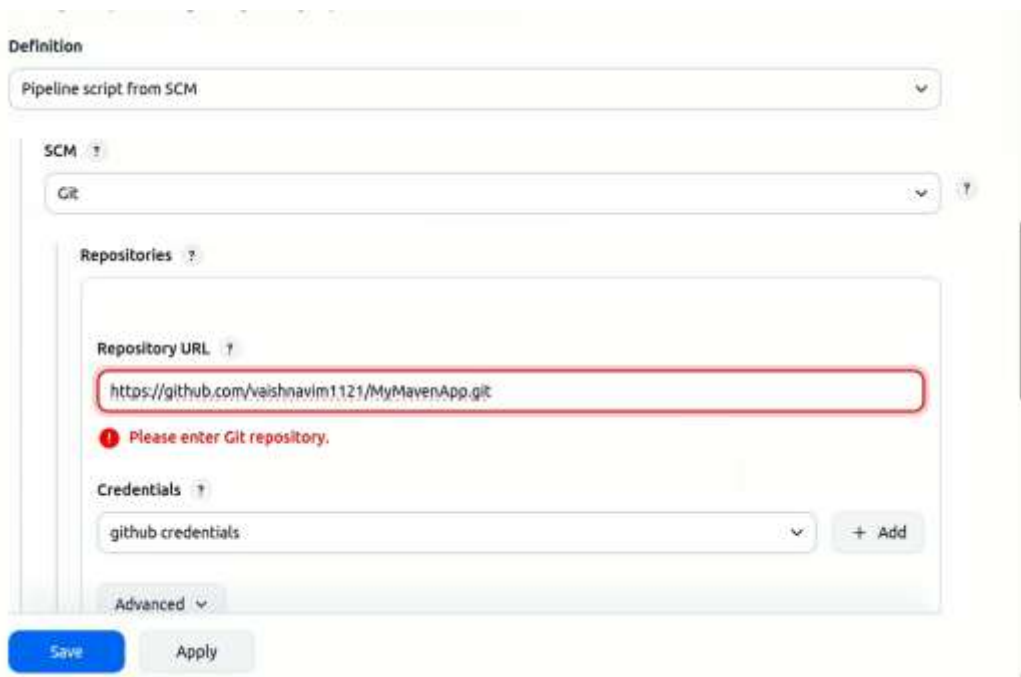
Enter an item name

Select an item type

-  **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

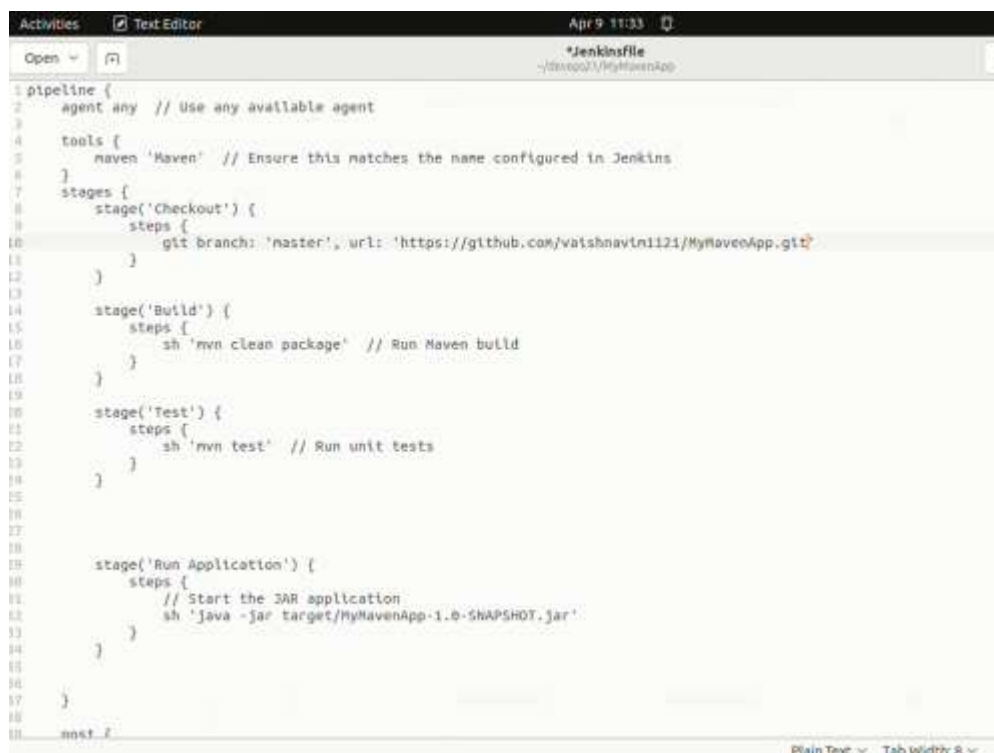
OK

This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “OK” to proceed to the job configuration page



The screenshot shows the Jenkins Pipeline Configuration page. Under the 'Definition' section, 'Pipeline script from SCM' is selected. In the 'SCM' section, 'Git' is chosen. The 'Repository URL' field contains 'https://github.com/vaishnavim1121/MyMavenApp.git', which is highlighted with a red border. Below it, a red error message states 'Please enter Git repository.' The 'Credentials' section shows 'github credentials' selected. At the bottom, there are 'Save' and 'Apply' buttons.

Now we need to add a jenkinsfile into our project which will consist of the operations that will occur when building the project in Jenkins
It includes step wise instructions guiding at each stage with right commands



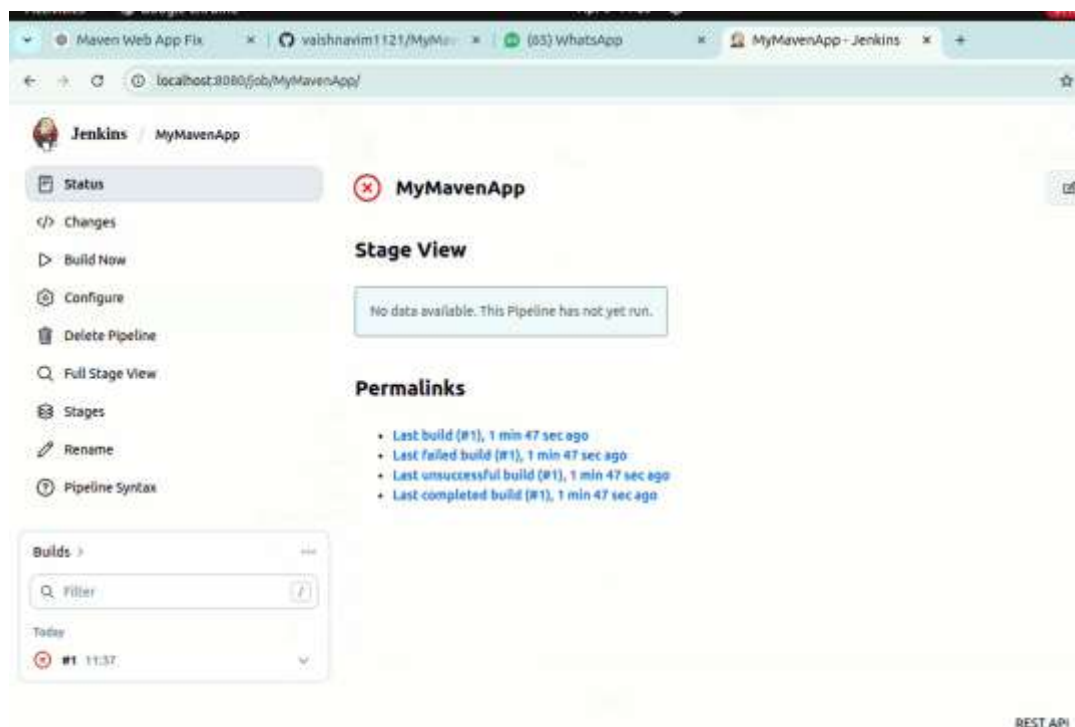
The screenshot shows a text editor displaying the content of a Jenkinsfile. The file defines a pipeline with four stages: Checkout, Build, Test, and Run Application. The Checkout stage uses the 'git' tool to checkout the 'master' branch. The Build stage runs 'mvn clean package'. The Test stage runs 'mvn test'. The Run Application stage runs 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'.

```

1 pipeline {
2   agent any // Use any available agent
3
4   tools {
5     maven 'Haven' // Ensure this matches the name configured in Jenkins
6   }
7   stages {
8     stage('Checkout') {
9       steps {
10        git branch: 'master', url: 'https://github.com/vaishnavim1121/MyMavenApp.git'
11      }
12    }
13    stage('Build') {
14      steps {
15        sh 'mvn clean package' // Run Maven build
16      }
17    }
18    stage('Test') {
19      steps {
20        sh 'mvn test' // Run unit tests
21      }
22    }
23    stage('Run Application') {
24      steps {
25        // Start the JAR application
26        sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
27      }
28    }
29  }
30 }

```

Now try building the project with Jenkins



The error occurs because there is no Jenkinsfile in the github repository. Thus we need to push the content to our project repository in order to run the project successfully.

Push the project again to your git account

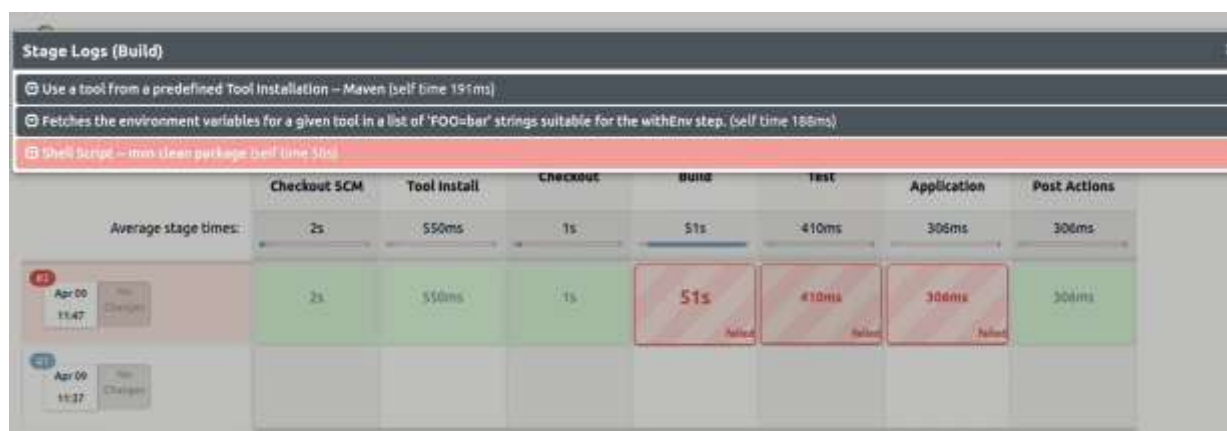
Now try building the project in Jenkins by clicking on build-now option.

MyMavenApp - Stage View

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Run Application	Declarative: Post Actions
Average stage times:	2s	550ms	1s	51s	410ms	306ms	306ms
#1 Apr 09 11:47 No Changes	2s	550ms	1s	51s failed	410ms failed	306ms failed	306ms
#1 Apr 09 11:37 No Changes							

Check the logs in order to rectify the error.

The error is due to version being outdated,



In order to correct it, edit the version that is mentioned in the pom.xml file.
 Change the version from 1.7 to 11 in both source and target tabs.
 After committing the changes, build again in the Jenkins

MyMavenApp - Stage View

